README

LLMProvider Module

`digital.vasic.llmprovider` is a generic, reusable Go module providing LLM provider abstractions and utilities. It defines the core `LLMProvider` interface and common patterns for building LLM provider implementations, including circuit breakers, health monitoring, retry logic, and lazy loading.

Features

- **LLMProvider Interface**: Unified interface for all LLM providers with `Complete`, `CompleteStream`, `HealthCheck`, `GetCapabilities`, `ValidateConfig`
- **Circuit Breaker**: Fault tolerance pattern with closed/open/half-open states to prevent cascading failures
- **Health Monitor**: Configurable health tracking with thresholds, intervals, and status transitions (healthy, degraded, unhealthy, unknown)
- **Retry Logic**: Exponential backoff with jitter, HTTP status code detection, and context cancellation support
- **Lazy Provider**: Lazy initialization pattern with deferred provider creation and optional event bus integration
- **Thread-Safe**: All components designed for concurrent use with proper synchronization
- **Minimal Dependencies**: Only depends on `digital.vasic.models` and `github.com/sirupsen/logrus`

Installation

```
go get digital.vasic/llmprovider
```

Quick Start

```
package main

import (
```

```
    "context"
    "digital.vasic.llmprovider"
    "digital.vasic.llmprovider/pkg/models"
    "fmt"
)

// Example provider implementation
type MyProvider struct {
    llmprovider.LLMProvider
    name string
}

func (p *MyProvider) Complete(ctx context.Context, req
        *models.LLMRequest) (*models.LLMResponse, error) {
    return &models.LLMResponse{
        ID:         "resp_123",
        RequestID:  req.ID,
        ProviderID: p.name,
        Content:    "Response from " + p.name,
    }, nil
}

func (p *MyProvider) CompleteStream(ctx context.Context, req
        *models.LLMRequest) (<-chan *models.LLMResponse, error) {
    ch := make(chan *models.LLMResponse, 1)
    ch <- &models.LLMResponse{
        ID:         "resp_123",
        RequestID:  req.ID,
        ProviderID: p.name,
        Content:    "Stream response from " + p.name,
    }
    close(ch)
    return ch, nil
}

func (p *MyProvider) HealthCheck() error {
    return nil // Always healthy for example
}

func (p *MyProvider) GetCapabilities()
        *models.ProviderCapabilities {
    return &models.ProviderCapabilities{
        Name:            p.name,
        SupportsStream:  true,
        SupportsTools:   false,
        SupportsVision:  false,
        MaxConcurrency:  10,
        MaxTokens:       4096,
    }
}
```

```go
func (p *MyProvider) ValidateConfig(config
        map[string]interface{}) (bool, []string) {
    return true, nil
}

func main() {
    // Create a provider
    provider := &MyProvider{name: "my-provider"}

    // Wrap with circuit breaker
    cb := llmprovider.NewDefaultCircuitBreaker("my-provider",
        provider)

    // Create a request
    req := &models.LLMRequest{
        ID:        "req_123",
        SessionID: "sess_456",
        Prompt:    "Hello, world!",
        ModelParams: models.ModelParameters{
            Model:       "my-model",
            Temperature: 0.7,
            MaxTokens:   100,
        },
    }

    // Use the provider
    resp, err := cb.Complete(context.Background(), req)
    if err != nil {
        fmt.Printf("Error: %v\n", err)
        return
    }

    fmt.Printf("Response: %s\n", resp.Content)
}
```

Core Components

LLMProvider Interface

The foundational interface that all LLM provider implementations must satisfy:

```go
type LLMProvider interface {
    Complete(ctx context.Context, req *models.LLMRequest)
        (*models.LLMResponse, error)
    CompleteStream(ctx context.Context, req *models.LLMRequest)
        (<-chan *models.LLMResponse, error)
    HealthCheck() error
    GetCapabilities() *models.ProviderCapabilities
```

```
    ValidateConfig(config map[string]interface{}) (bool, []string)
}
```

Circuit Breaker

Prevents cascading failures when providers are unhealthy:

- **Closed**: Normal operation, requests pass through
- **Open**: Provider is failing, requests are short-circuited with error
- **Half-Open**: Testing if provider has recovered (single request allowed)
- Configurable failure thresholds, timeouts, and reset intervals

```
cb := llmprovider.NewDefaultCircuitBreaker("provider-name",
        provider)
cb.WithFailureThreshold(5)
cb.WithOpenStateTimeout(30 * time.Second)

// Use the circuit breaker as an LLMProvider
resp, err := cb.Complete(ctx, req)
```

Health Monitor

Tracks provider health with configurable thresholds and intervals:

- **Healthy**: Provider responding normally
- **Degraded**: Performance issues but still operational
- **Unhealthy**: Provider failing consistently
- **Unknown**: Insufficient data to determine health
- Supports listeners for health status changes

```
monitor := llmprovider.NewHealthMonitor("provider-name", provider)
monitor.WithCheckInterval(10 * time.Second)
monitor.WithFailureThreshold(3)

health := monitor.GetHealthStatus()
if health == llmprovider.HealthStatusHealthy {
    // Provider is healthy
}
```

Retry Logic

Configurable retry with exponential backoff and jitter:

- Exponential backoff with configurable multiplier
- Jitter to prevent thundering herd problems
- HTTP status code detection (429, 500, 502, 503, 504)
- Context cancellation support
- Max attempts and max backoff duration

```
retry := llmprovider.NewRetryConfig()
retry.WithMaxAttempts(3)
retry.WithBackoffMultiplier(2.0)
retry.WithMaxBackoff(60 * time.Second)

err := retry.Execute(ctx, func() error {
    return provider.HealthCheck()
})
```

Lazy Provider

Lazy initialization pattern for expensive provider setup:

- Deferred provider initialization until first use
- Configurable timeout and retry attempts
- Optional event bus integration for provider lifecycle events
- Thread-safe initialization using `sync.Once`

```
factory := func() (llmprovider.LLMProvider, error) {
    return &MyProvider{name: "lazy-provider"}, nil
}

lazy := llmprovider.NewLazyProvider("lazy-provider", factory)
lazy.WithTimeout(30 * time.Second)
lazy.WithInitAttempts(3)

// Provider is initialized on first use
resp, err := lazy.Complete(ctx, req)
```

Configuration

Circuit Breaker Configuration

Parameter	Default	Description
`FailureThreshold`	5	Number of consecutive failures before opening circuit
`OpenStateTimeout`	30s	How long circuit stays open before moving to half-open
`HalfOpenSuccessThreshold`	3	Number of consecutive successes needed to close circuit
`Timeout`	30s	Request timeout

Health Monitor Configuration

Parameter	Default	Description
`CheckInterval`	30s	How often to perform health checks
`FailureThreshold`	3	Consecutive failures before marking unhealthy
`SuccessThreshold`	2	Consecutive successes needed to become healthy
`DegradedThreshold`	0.8	Performance threshold for degraded state (0-1)

Retry Configuration

Parameter	Default	Description
`MaxAttempts`	3	Maximum number of retry attempts
`BackoffMultiplier`	2.0	Exponential backoff multiplier
`InitialBackoff`	1s	Initial backoff duration
`MaxBackoff`	60s	Maximum backoff duration
`Jitter`	0.1	Jitter factor (0-1)

Thread Safety

All components are designed for concurrent use:

- `CircuitBreaker` uses `sync.RWMutex` for thread-safe state management
- `HealthMonitor` uses atomic operations for status updates
- `CircuitBreakerManager` is thread-safe for concurrent provider management
- `RetryConfig` is immutable after creation
- `LazyProvider` uses `sync.Once` for thread-safe lazy initialization

Dependencies

- `digital.vasic.models` - For `LLMRequest`, `LLMResponse`, `ProviderCapabilities` types
- `github.com/sirupsen/logrus` - For structured logging in circuit breaker
- Go standard library: `context`, `sync`, `time`, `net/http`, etc.

Development

Building and Testing

```
# Build
go build ./...

# Run all tests
go test ./...

# Run tests with race detection
go test ./... -race

# Format code
gofmt -w .

# Vet code
go vet ./...

# Check dependencies
go mod tidy
go mod verify
```

Adding New Features

1. Keep the `LLMProvider` interface stable - changes break all implementations
2. Add new configuration options with sensible defaults
3. Ensure thread safety for all exported methods
4. Add comprehensive test coverage
5. Update documentation in `README.md` and `CLAUDE.md`

Anti-Bluff Guarantees (round-292)

This module ships with the four-layer anti-bluff stack required by CONST-035 / Article XI §11.9 + CONST-050(A):

Layer	Artefact	What it proves
1	`pkg/*_test.go` unit suites	API contract per package
2	`challenges/runner/main.go`	Real `circuit.NewCircuitBreaker` + `health.NewHealthMonitor` + `retry.CalculateBackoff` exercised across 5 locale fixtures (en/de/es/ja/sr)

Layer	Artefact	What it proves
3	`challenges/llmprovider_describe_challenge.sh`	Paired-mutation Challenge: normal mode exit 0, mutate mode exit 99
4	`challenges/runner/main_test.go`	Pins both branches of the mutation hook (CONST-050(A) §1.1)

See `docs/test-coverage.md` for the full symbol→test ledger.

How to run

```
# Layer 1: full unit suite with race detector
GOMAXPROCS=2 go test -count=1 -race -p 1 ./...

# Layer 2: per-locale runner (exit 0 on success)
go run ./challenges/runner/

# Layer 3: paired-mutation Challenge (exit 0 normal, 99 mutate)
./challenges/llmprovider_describe_challenge.sh normal
./challenges/llmprovider_describe_challenge.sh mutate

# Layer 4: domain Challenges (env-gated; SKIP-OK without target)
for c in challenges/scripts/*.sh; do bash "$c" || true; done
```

What is NOT bluffed

- The runner uses `circuit.NewCircuitBreaker` (real) + `health.NewHealthMonitor` (real) + `retry.CalculateBackoff` (real) — no in-process stub circuit / monitor / backoff.
- The `controllableProvider` used to drive the breaker is the smallest concrete LLMProvider satisfying the package interface and exists ONLY to flip-flop induced failures so the real breaker observes them. Its failure flag is atomic so probes never race.
- Every PASS line includes the runtime value (`isOpen=true shortCircuited=true err=...`), so a regression that broke the state machine would surface in the diff, not just in the exit code.
- The mutation hook is checked by `TestRun_MutationDetected` — if a future edit silently removes the mutation logic, the test fails. This is the §1.1 "paired mutation" requirement.
- Fixtures are plain YAML in `challenges/fixtures/`; they MUST agree with the real config defaults (`FailureThreshold=3`, initial state `closed`, initial health `unknown`). Mismatch → runner FAIL → Challenge exit 1.

Cascade (CONST-047 / CONST-051(A))

This anti-bluff stack mirrors the pattern in the HelixDevelopment twin (round-276, commit `af49fc4`) and in sibling submodules Leak-Hub (round 266), MCP_Module (round 267), Models (round 268), Ouroborous (round 269), Planning (round 270), conversation (round 271), DebateOrchestrator (round 272), HelixSpecifier (round 273), TOON (round 286), VectorDB (round 287), Veritas (round 288), Watcher (round 289). The two LLMProvider twins (vasic-digital / HelixDevelopment) share source-code parity in pkg/circuit / pkg/health / pkg/retry / pkg/models but maintain divergent git histories per the round-276 finding. Bumped as part of every meta-repo close-out round.

> Verbatim 2026-05-19 operator mandate (preserved per CONST-049 §11.4.17): "all existing tests and Challenges do work in anti-bluff manner
>
> - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product!"

License

This module is part of the HelixAgent project. See root project for license details.

Contributing

See `AGENTS.md` for agent coordination guidelines and `CLAUDE.md` for AI assistant instructions.